

AN1.8 - Evidence Hardening + Output Integrity (A6)

Sprint: AT24 AI Agents - Agent Framework Foundation **Step:** A6 - Evidence Hardening + Output Integrity **Depends on:** AF-v1 contracts, Tool Registry (A2), Persistence (A3), Runtime (A4), Supervisor (A5 / G05) **Gate:** G06 - trustworthiness is structural, not prompt-dependent.

The invariant: an agent conclusion is not trustworthy merely because the agent produced it. Before a run may be succeeded, its output must pass a **deterministic, LLM-independent** integrity gate and prove evidence lineage back to a real AT24 capability. Failure is a deterministic terminal `failed / output_integrity`.

1. What was built - `services/agent-framework/integrity/`

Server-only. Pure functions - the runtime passes in the persisted trace; no I/O, no LLM, no dependency on the Supervisor's or a specialist's own discipline.

File	Role
<code>evidence-lineage.ts</code>	<code>buildLineage(output, trace, registry)</code> - traverses <code>output.evidenceIds</code> -> <code>AgentEvidence</code> -> <code>AgentToolCall</code> -> <code>AgentStep</code> -> <code>AgentRun</code> -> a registered tool and returns the chain (<code>LineageLink[]</code> , each ending at the tool's <code>wraps</code> pointer - the real AT24 service) plus every gap. <code>outputEvidenceIds(output)</code> extracts the cited ids.
<code>output-integrity.ts</code>	<code>checkOutputIntegrity({ output, trace, definition, registry })</code> -> <code>IntegrityResult</code> - the gate.
<code>index.ts</code>	server-only barrel.

1.1 Runtime wiring

AgentRuntime's "plan exhausted -> succeeded" branch now:

```
synthesise (Supervisor)
  | append "output" step
  OUTPUT INTEGRITY CHECK (checkOutputIntegrity)
    | append "evaluation" step { passed, violations, lineage }
    |-- passed -> patchRun { status: "succeeded", output }
    \-- failed -> patchRun { status: "failed", output (kept for forensics),
                          errorCode: "output_integrity",
                          errorMessage: "<code>: <msg> | ..." }
```

running -> failed is a permitted AF-v1 transition. The rejected output **is still persisted** (so a developer can see what the agent tried to conclude), but status is the authority - **succeeded is the only status where AgentRun.output is trustworthy**. Every run now carries an `evaluation` step recording the integrity verdict + full lineage.

2. What the integrity check verifies

#	Check	Violation code
1	output is a plain object (not null / array / primitive); <code>summary</code> is a string when present	<code>malformed_output</code>

#	Check	Violation code
2	every id in <code>output.evidenceIds</code> exists in this run's evidence	<code>foreign_evidence_id</code>
3	full lineage: each cited evidence -> a real step, a real tool call in this run, a registered tool	<code>broken_lineage</code>
4	a <code>resolved: true</code> conclusion actually cites evidence, the run captured evidence, and <code>basis</code> is non-empty	<code>unsupported_claim</code>
5	each cited evidence row re-passes the <code>AF-v1 validateAgentEvidence</code> contract (provenance producer + ISO <code>retrievedAt</code> , type, required fields, relevance/confidence in [0,1])	<code>invalid_evidence</code>
6	no credential in an evidence row's <code>provenance</code> (<code>api_key</code> / <code>secret</code> / <code>bearer</code> / <code>authorization</code> / <code>-----BEGIN</code> / <code>sk-...</code>)	<code>sensitive_data</code>
7	a constrained agent (<code>autonomyLevel</code> < 2 - every v1 agent) emits no trade-instruction field at any depth (<code>entry</code> , <code>entryZone</code> , <code>stopLoss</code> , <code>takeProfit</code> , <code>target</code> , <code>positionSize</code> , <code>side</code> , <code>signal</code> , <code>orderType</code> , <code>recommendation</code> , <code>action</code> , ...)	<code>forbidden_trading_field</code>
8	a constrained agent's string values (except the whitelisted <code>disclaimer</code> / <code>note</code> keys) contain no signal / performance-claim language (<code>buy</code> , <code>sell</code> , <code>go long/short</code> , <code>win-rate</code> , <code>probability of profit</code> , <code>NN% chance</code> , <code>guaranteed</code>)	<code>forbidden_signal_language</code>

Checks 7-8 are **structural**: they apply to any `autonomyLevel` < 2 agent regardless of type or specialist. The Market Intelligence specialist's own "promise not to emit trading instructions" is no longer the safety boundary - this gate is.

Not covered by A6 (noted): deep semantic "unsupported claim" detection for free-form LLM narrative. There is no LLM narrative in the deterministic synthesis today; when a presenter LLM is added (AN1.1 sec 16), it will reuse the existing `ai-response-integrity.service.ts` claim-check pattern feeding this same gate.

3. G06 proof - `npm run validate:agent-integrity` -> 21 passed, 0 failed

3.1 Valid case

```
real tool (market.snapshot) -> real evidence -> valid provenance
-> output { resolved, bias, basis, evidenceIds: ["ev_1"], disclaimer }
-> checkOutputIntegrity -> PASS
-> lineage.complete = true, links[0].capability = <market.snapshot wraps pointer>
```

3.2 Adversarial cases - each deterministically FAILS

case	code
output is a string / an array	<code>malformed_output</code>

case	code
summary is a number	malformed_output
resolved: true but evidenceIds: []	unsupported_claim
resolved: true but basis: []	unsupported_claim
cites ev_FOREIGN (not in run)	foreign_evidence_id
evidence row's runId != the run	broken_lineage
evidence -> toolCallId not in the run	broken_lineage
evidence -> tool totally.made.up (unregistered)	broken_lineage
evidence provenance producer: ""	invalid_evidence
evidence provenance carries api_key: "sk-..."	sensitive_data
output.entryZone = [...]	forbidden_trading_field
output.setup.stopLoss = ... (nested)	forbidden_trading_field
output.summary = "strong buy signal"	forbidden_signal_language
output.note2 = "expected win-rate 72%"	forbidden_signal_language

Plus: the disclaimer key **may** say "not a buy or sell recommendation"; an autonomyLevel: 2 agent **may** carry positionSize; buildLineage reports gaps for a foreign id and a missing step.

3.3 Runtime integration

- **RUNTIME: a specialist emitting a forbidden field** - a planner whose synthesizeOutput deliberately returns { entry, stopLoss, summary: "strong buy" } drives a real run to status: "failed", errorCode: "output_integrity", errorMessage matching forbidden_trading_field|forbidden_signal_language, with an evaluation step status: "error", output.passed: false.
- **RUNTIME: a compliant run still reaches succeeded** - the real MI E2E: succeeded with integrity PASS, 5 evidence; the evaluation step's output.passed: true.

3.4 Regression - no gate lost

suite	before	after
validate:agent-contracts	38/0	38/0
validate:agent-tools	20/0	20/0
validate:agent-run-persistence	17/0	17/0
validate:agent-runtime	9/0	9/0 (step-kind assertion updated for the new evaluation step)
validate:agent-supervisor	11/0	11/0
validate:agent-integrity	-	21/0

4. TypeScript

npx tsc --noEmit: **0 errors** in services/agent-framework/**, types/agent-framework/**, scripts/validate-agent-*.ts. Repo-wide 77, all in the stale generated .next/dev/types/validator.ts (gitignored, pre-existing).

5. Files changed

Added (4):

```
frontend/services/agent-framework/integrity/evidence-lineage.ts
frontend/services/agent-framework/integrity/output-integrity.ts
frontend/services/agent-framework/integrity/index.ts
frontend/scripts/validate-agent-integrity.ts
```

Modified (3):

frontend/services/agent-framework/runtime/agent-runtime.ts	integrity gate before "succeeded"; adds an "evaluation" step-kind assertion + the new "evaluation" step-kind assertion + the new "evaluation" step-kind assertion
frontend/scripts/validate-agent-runtime.ts	
frontend/package.json	+ "validate:agent-integrity"

Not touched: Prisma schema, contracts, tool layer, Supervisor, tick() / LimitEnforcer / RunTracer / authorizer internals, legacy services/agents/* + /dashboard/agents. No API routes. No new agents / tools / engines / UI.

6. Non-goals honoured

No Research / Strategy agent - no autonomous trading - no new tools - no new intelligence engine - no new backtest engine - no UI. A6 is foundation: making the runtime's output *trustworthy* before real specialist agents attach.

7. Status

A6 complete. The output-integrity gate is deterministic, LLM-independent, and enforced by the runtime before any `succeeded`. Evidence lineage is traceable end to end. The Market Intelligence conclusion is now governance-bounded by the framework, not by specialist good behaviour.

Next per AN1.1 sec 7: A7 (Memory contract + `MemoryGateway`), A8 (Permissions/Guardrails deepening), A9 (Credit ledger), A10 (Evaluation + Observability), then A11-A13 (the three real agents).

G06 review requested.

End AN1.8.